ABSTRACT

ABSTRACT

Kisan Mithr is a smart agricultural support system developed to help farmers access timely and reliable information for better farming decisions. The project addresses the problem of limited access to crop guidance, weather updates, market prices, and government schemes in rural areas by providing a unified and user-friendly platform. It uses modern web technologies along with artificial intelligence, allowing farmers to interact through text, voice, and image-based inputs, making the system accessible even to users with minimal technical knowledge.

The system is built as a full-stack application with AI-driven recommendations, real-time data integration, and features such as crop advisory, disease detection, and scheme awareness. It also includes an admin module for efficient user and data management. The results demonstrate improved decision-making, increased awareness, and reduced dependency on multiple information sources. Overall, Kisan Mithr bridges the gap between traditional farming and modern technology, contributing to better productivity, sustainability, and farmer empowerment.

INTRODUCTION

INTRODUCTION

Kisan Mithr is a smart agricultural platform designed to help farmers get all important farming information in one place. Many farmers face problems like lack of proper guidance, changing weather, price fluctuations, and limited awareness about government schemes. To solve this, Kisan Mithr provides a simple and user-friendly system where farmers can easily access crop advice, weather updates, market prices, and other useful services.

The platform uses modern technologies like artificial intelligence to make interaction easy through text, voice, and images. It also offers features such as crop disease detection, real-time updates, and personalized suggestions. Overall, Kisan Mithr helps farmers make better decisions, improve productivity, and adopt modern farming practices in a simple and effective way.

MODULE DESCRIPTION

MODULE DESCRIPTION

1) **User Model** – This model stores all farmer-related information such as name, location, contact details, and login credentials. It helps in managing user authentication and providing personalized services based on the user's profile.

2) **Admin Model** – This model is used to manage administrator details and system controls. It allows admins to monitor users, update content, and ensure smooth functioning of the platform.

3) **Crop Advisory Model** – This model provides crop-related recommendations based on factors like season, soil type, and location. It helps farmers choose suitable crops and follow best farming practices.

4) **Weather Model** – This model integrates real-time weather data from external sources. It provides updates on temperature, rainfall, and climate conditions to help farmers plan their activities effectively.

5) **Market Price Model** – This model displays current market prices of various crops. It enables farmers to make better decisions regarding selling their produce at the right time.

6) **Disease Detection Model** – This AI-based model analyzes images of crops uploaded by farmers to detect possible diseases. It helps in early identification and suggests preventive measures.

7) **Government Schemes Model** – This model provides information about various government schemes and benefits available to farmers. It helps users stay informed and take advantage of financial support programs.

8) **AI Assistant Model** – This model processes user queries in text or voice format using artificial intelligence. It provides instant responses, guidance, and suggestions related to farming.

FEASIBILITY STUDY

FEASIBILITY STUDY

The feasibility study of Kisan Mithr evaluates whether the system is practical, cost-effective, and useful for real-world implementation. The project is designed using easily available technologies, making it simple to develop and deploy. It requires minimal cost due to the use of open-source tools and does not need expensive hardware. The system is also user-friendly, allowing farmers to easily access important agricultural information. Overall, Kisan Mithr is a feasible solution that can effectively support farmers and improve agricultural practices.

TYPES OF FEASIBILITY:

1) **Technical Feasibility** –
   - This checks whether the required technologies like web development tools, databases, and AI features are available and suitable for building the system.
   - In Kisan Mithr, all technologies used are easily accessible and can be implemented without any major technical difficulty.

2) **Economic Feasibility** –
   - This evaluates the overall cost of developing and maintaining the project.
   - Kisan Mithr is cost-effective because it uses mostly open-source tools and does not require expensive hardware, making it affordable for implementation.

3) **Operational Feasibility** –
   - This determines how easy it is for users, especially farmers, to use the system in real life.
   - Kisan Mithr is designed with a simple interface and supports voice, text, and image inputs, making it user-friendly and practical.

4) **Social Feasibility** –
   - This analyzes how beneficial the system is to society. Kisan Mithr positively impacts farmers by providing useful information, improving productivity, and encouraging the use of technology in agriculture.

SOFTWARE DEVELOPMENT LIFE CYCLE

SDLC

Requirement Analysis
System Design
Implementation
Testing
Deployment
Maintenance
SDLC
Kisan Mithr Project

SDLC

System Development Life Cycle Model (SDLC Model)

The Software Development Life Cycle (SDLC) is a structured process used to design, develop, test, and deploy a software system efficiently. In the Kisan Mithr project, SDLC is followed to ensure that the system is developed in a systematic and organized manner, meeting user requirements and maintaining quality at every stage. It helps in proper planning, execution, and maintenance of the project.

1. Requirement Analysis

In this phase, the requirements of the Kisan Mithr system are identified and analyzed. The needs of farmers are studied, such as accessing crop advice, weather updates, market prices, and government schemes. This phase ensures that the system is developed based on real user needs and expectations.

2. System Design

In the design phase, the overall structure of the system is planned. This includes designing the frontend interface, backend architecture, database models, and API integrations. A clear design helps in building the system efficiently and avoids confusion during development.

3) Implementation

This phase involves the actual development of the project. All modules such as user interface, backend services, AI features, and database connectivity are coded. Each feature of Kisan Mithr is developed step-by-step to ensure proper functionality.

4) Testing

After development, the system is tested to identify and fix errors. Different types of testing are performed to ensure that all features work correctly and the system is reliable. This phase improves the quality and performance of the application.

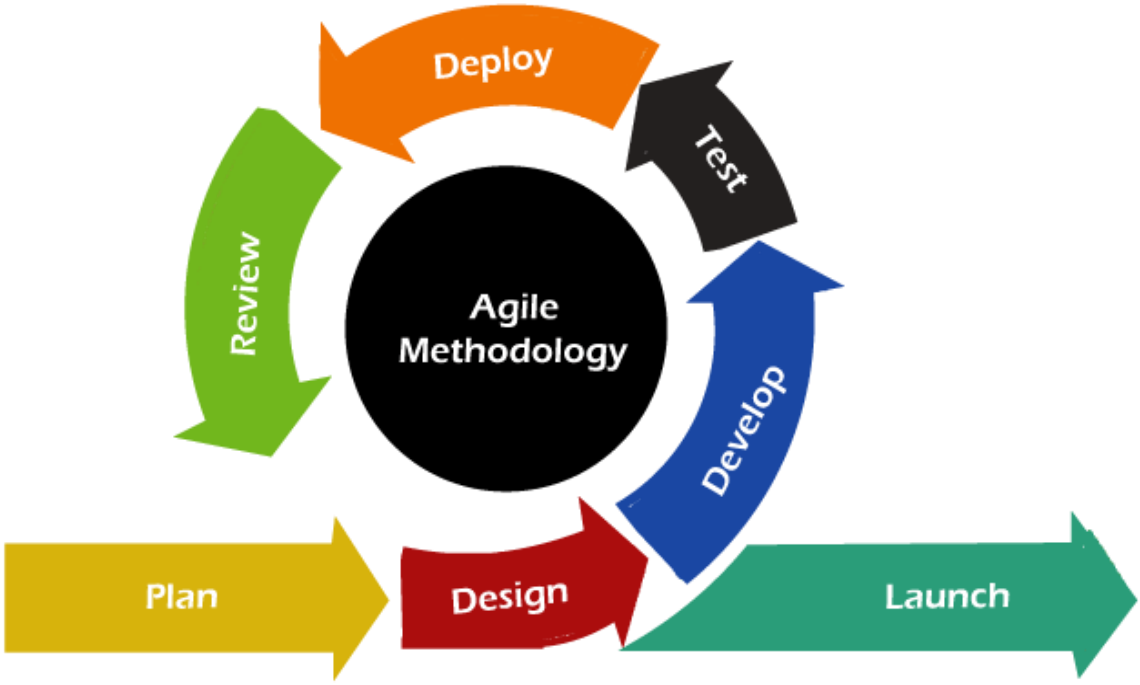
5) Deployment

In this phase, the Kisan Mithr system is made available to users. The application is deployed on a server so that farmers can access it through web browsers on mobile or desktop devices.

6) Maintenance

The final phase involves maintaining and updating the system. Bugs are fixed, new features are added, and improvements are made based on user feedback. This ensures that the system remains useful and up-to-date over time.

AGILE MODEL


Deploy
Test
Review
Agile Methodology
Develop
Plan
Design
Launch

SYSTEM REQUIREMENTS

HARDWARE REQUIREMENTS:

Processor: Intel i3 or above

RAM: Minimum 4 GB (8 GB recommended)

Operating System: Windows / Linux / macOS / Android

Storage: At least 20 GB free space

Device: Desktop / Laptop / Smartphone

Internet: Stable internet connection

SOFTWARE REQUIREMENTS:

Frontend: JavaScript (JSX), React JS, Tailwind CSS

Backend: JavaScript (Node.js), Express

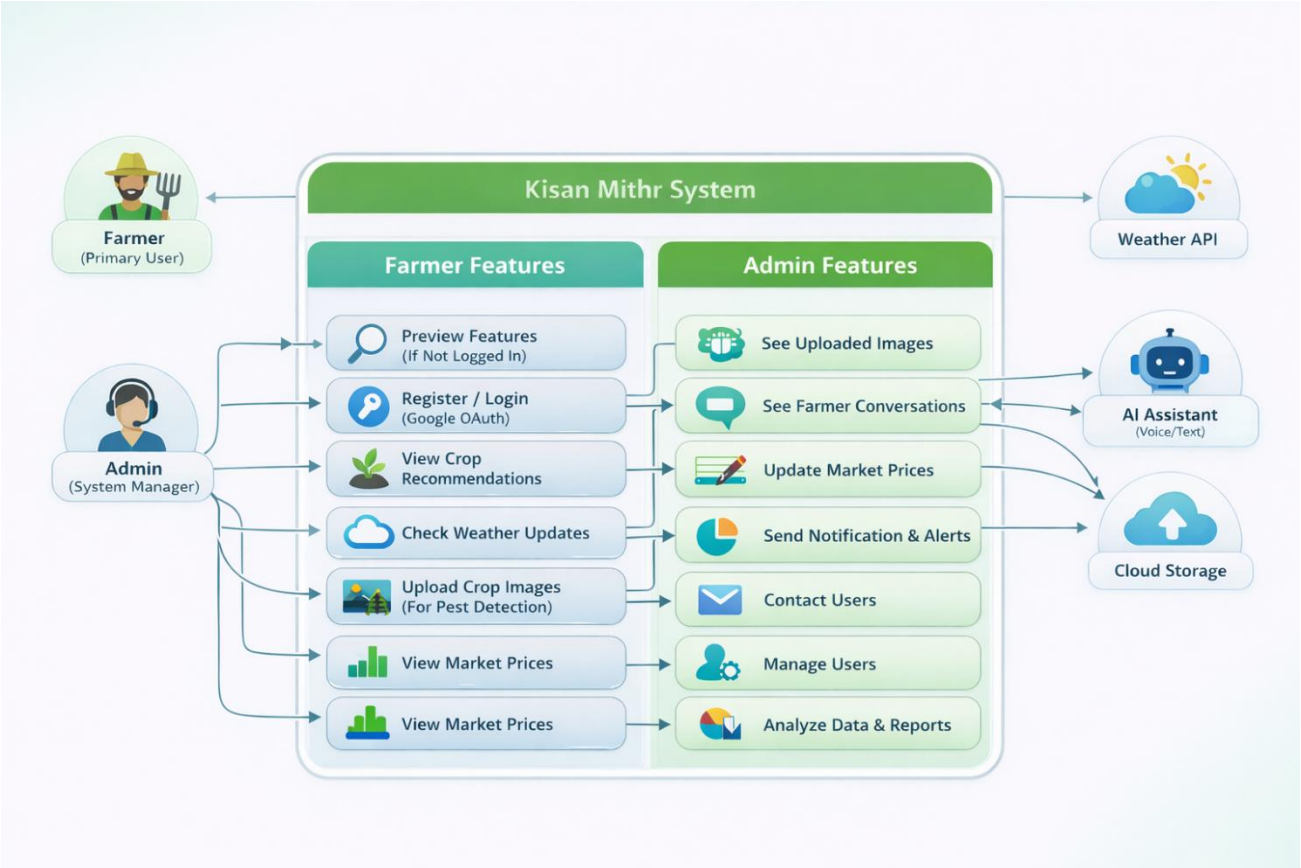
Database: MongoDB, Cloudinary

API'S:

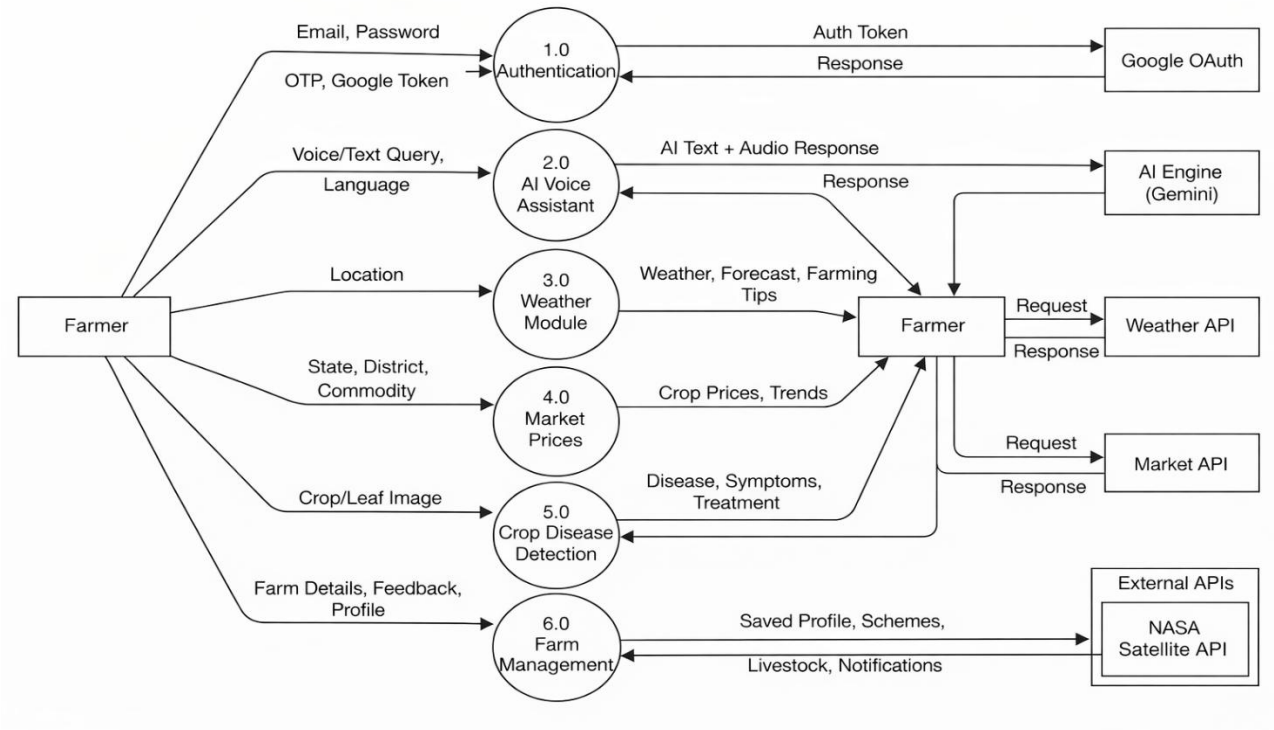
- Groq (LLaMA)
- ElevenLabs, Edge TTS, Sarvam AI TTS
- NASA EONET / APOD
- OpenWeatherMap
- Data.gov.in
- EmailJS
- Google Gemini AI
- Google OAuth

SYSTEM DESIGN

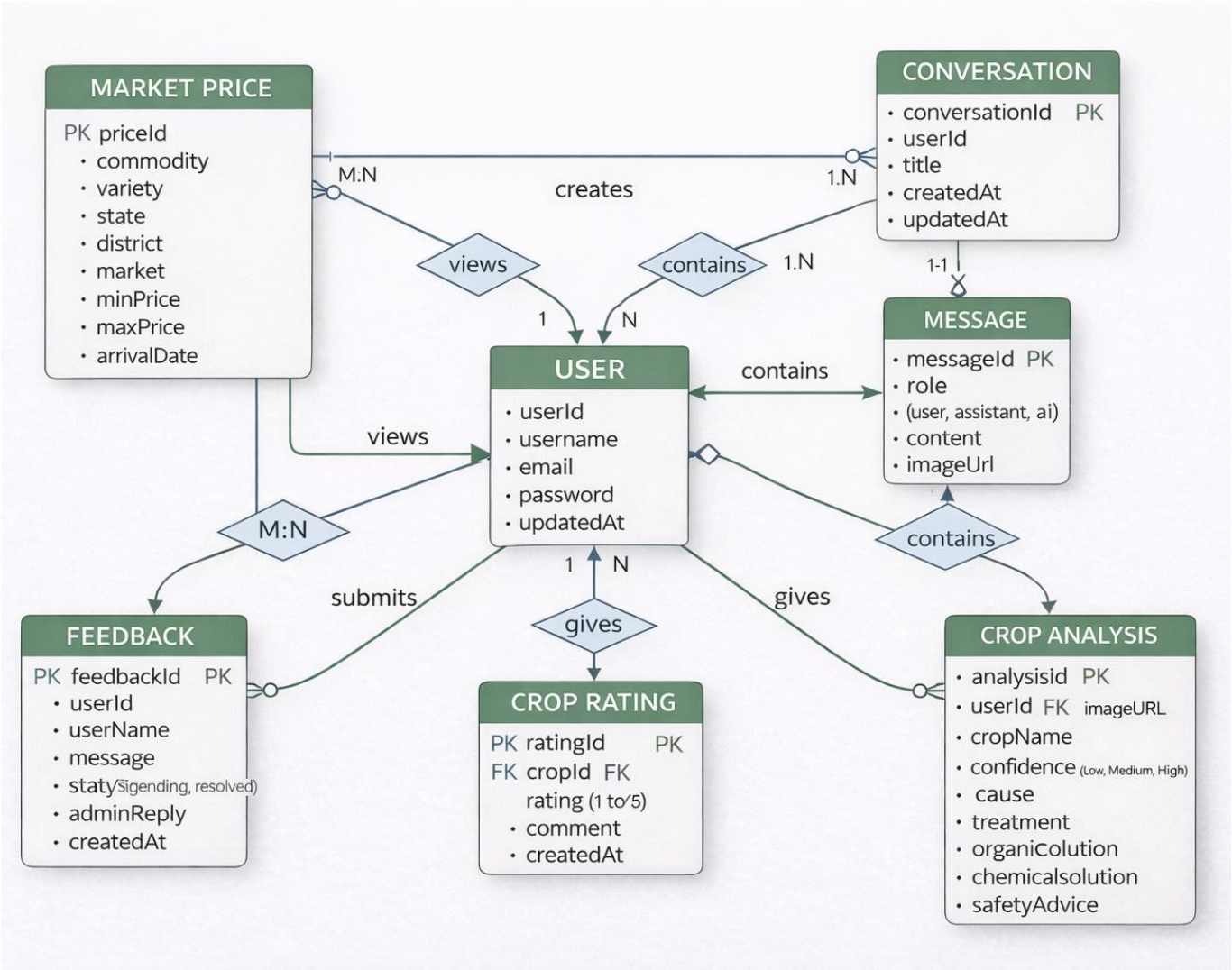
A) USE-CASE DIAGRAM



B) DATAFLOW DIAGRAM



C) E-R DIAGRAM


MARKET PRICE
PK priceId
• commodity
• variety
• state
• district
• market
• minPrice
• maxPrice
• arrivalDate
CONVERSATION
• conversationId PK
• userId
• title
• createdAt
• updatedAt
M.N
creates
1.N
views
contains
1.N
1-1
1
N
MESSAGE
• messageId PK
• role
• (user, assistant, ai)
• content
• imageUrl
USER
• userId
• username
• email
• password
• updatedAt
contains
views
M:N
contains
1
N
submits
gives
gives
FEEDBACK
PK feedbackId PK
• userId
• userName
• message
• statysigending, resolved)
• adminReply
• createdAt
CROP RATING
PK ratingId PK
FK cropId FK
rating (1 to 5)
• comment
• createdAt
CROP ANALYSIS
• analysisid PK
• userId FK imageURL
• cropName
• confidence (Low, Medium, High)
• cause
• treatment
• organicolution
• chemicalsolution
• safetyAdvice

D) DATABASE COLLECTIONS

1. Users Collection Schema

Field Name	Data Type	Description
_id	ObjectId	Primary key, auto-generated
username	String	Display name of the farmer
email	String	Email address (unique, lowercase)
password	String	Hashed password (required if not Google user)
googleId	String	Google OAuth unique identifier
googleName	String	Name from Google account
googleMail	String	Email from Google account
googlePhoto	String	Profile photo URL from Google
emailOtp	String	OTP code for email verification
emailOtpExpiry	String	Expiry timestamp of the OTP
profilePicture	String	URL of profile picture or avatar
resetPasswordToken	String	Token for password reset
resetPasswordExpires	Date	Expiry time of reset token
about	String	Short bio of the user
isVerified	Boolean	Whether email is verified (default: false)
role	String	User role (default: "Farmer")
agreed	Boolean	Terms and conditions acceptance (default: false)
createdAt	String	Account creation timestamp (IST)

2. Conversations Collection Schema

Field Name	Data Type	Description
_id	ObjectId	Primary key, auto-generated
userId	ObjectId (ref: Users)	Foreign key referencing the User
title	String	Conversation title (default: "New Chat")
messages	Array of Message objects	Embedded sub-documents (see below)
createdAt	Date	Auto-generated timestamp
updatedAt	Date	Auto-generated timestamp

3. Crop Analyses Collection Schema

Field Name	Data Type	Description
_id	ObjectId	Primary key, auto-generated
userId	ObjectId (ref: Users)	Foreign key referencing the User
imageURL	String	Uploaded crop image URL
cropName	String	Identified crop name
disease	String	Detected disease name
confidence	String (enum: Low, Medium, High)	AI confidence level
symptoms	String	Observed symptoms description
cause	String	Probable cause of disease
treatment	String	Recommended treatment
prevention	String	Prevention advice
organic_solution	String	Organic remedy suggestion
chemical_solution	String	Chemical remedy suggestion
safety_advice	String	Safety precautions
disclaimer	String	AI disclaimer text
status	String (enum: pending, verified, incorrect)	Verification status
date	Date	Analysis date

4. Market Prices Collection Schema

Field Name	Data Type	Description
_id	ObjectId	Primary key, auto-generated
commodity	String	Crop/commodity name
variety	String	Variety of the commodity
state	String	State name
district	String	District name
market	String	Market/mandi name
min_price	Number	Minimum price (₹/quintal)
max_price	Number	Maximum price (₹/quintal)
modal_price	Number	Modal/average price (₹/quintal)
arrival_date	String	Date of price record

5. Notifications Collection Schema

Field Name	Data Type	Description
_id	ObjectId	Primary key, auto-generated
title	String	Notification title
message	String	Notification content
region	String	Target region
cropType	String	Related crop type
isRead	Boolean	Read status (default: false)
senderName	String	Name of sender
createdBy	ObjectId (ref: Admin)	Admin who created it
createdAt	Date	Auto-generated timestamp
updatedAt	Date	Auto-generated timestamp

6. Image Collection Schema

Field Name	Data Type	Description
_id	ObjectId	Primary key, auto-generated
farmerId	ObjectId (ref: Users)	Foreign key referencing the User
imageUrl	String	Stored image URL
purpose	String	Purpose (e.g., disease_detection, profile_picture)
status	String (enum: active, inappropriate, deleted)	Moderation status
createdAt	Date	Upload timestamp
updatedAt	Date	Last update timestamp

CODING

----- HOMEPAGE.JSX -----

```
import { useState, useEffect } from "react";
import { useNavigate } from "react-router-dom";
import Header from "./header/Header";
import heroBG from "../assets/hero-bg.jpg";
import useTranslation from "../hooks/useTranslation";
import { Mic, CloudSun, LineChart, Leaf, ArrowRight, MessageSquare, Send, User,
Mail, Bot } from "lucide-react";
import axios from "axios";
import { toast } from "react-toastify";
import useUserStore from "../store/useUserStore";
import Footer from "./Footer";

const HomePage = () => {
  const [isScrolled, setIsScrolled] = useState(false);
  const navigate = useNavigate();
  const { t } = useTranslation();
  const { user } = useUserStore();

  const [feedback, setFeedback] = useState({
    name: user?.username || user?.googleName || "",
    message: ""
  });
  const [isSubmitting, setIsSubmitting] = useState(false);

  const handleFeedbackSubmit = async (e) => {
    e.preventDefault();
    if (!feedback.message.trim()) {
      toast.warning("Please enter your message or complaint.");
      return;
    }

    setIsSubmitting(true);
    try {
      await axios.post(`${process.env.REACT_APP_API_URL}/api/feedback`, {
        userName: feedback.name || "Anonymous Farmer",
        message: feedback.message,
        userId: user?.id || user?._id
      });
      toast.success("Thank you! Your feedback has been sent to the admin.");
      setFeedback({ ...feedback, message: "" });
    } catch (err) {
      console.error("Feedback error:", err);
      toast.error("Failed to send feedback. Please try again later.");
```

```
  } finally {
    setIsSubmitting(false);
  }
};

useEffect(() => {
  const handleScroll = () => {
    if (window.scrollY > 50) {
      setIsScrolled(true);
    } else {
      setIsScrolled(false);
    }
  };

  window.addEventListener("scroll", handleScroll);
  return () => window.removeEventListener("scroll", handleScroll);
}, []);

return (
  <div className="bg-zinc-950 min-h-screen font-sans selection:bg-green-500/30">
    <Header />

    {/* Hero Section */}
    <div className="relative h-[100dvh] w-full flex flex-col justify-center overflow-
hidden">

      {/* Background Image with Parallax & Darkening Overlay */}
      <div
        className="absolute inset-0 z-0 bg-cover bg-center transform scale-105
transition-transform duration-[20s] ease-out"
        style={{ backgroundImage: `url(${heroBG})` }}
      />
      <div className="absolute inset-0 z-0 bg-gradient-to-b from-black/80 via-
black/50 to-zinc-950 backdrop-blur-[2px]" />

      {/* Hero Content */}
      <div className="relative z-10 max-w-7xl mx-auto px-6 md:px-12 flex flex-col
items-center md:items-start text-center md:text-left mt-16 md:mt-24">

        <div className="inline-flex items-center gap-2 px-4 py-2 rounded-full border
border-green-500/30 bg-green-500/10 backdrop-blur-md mb-8 animate-fade-in-
down">
          <span className="w-2 h-2 rounded-full bg-green-400 animate-
pulse"></span>
```

```
          <span className="text-sm font-medium tracking-wide text-green-300
uppercase">
            {t("welcome")}
          </span>
        </div>

        <h1 className="text-5xl md:text-7xl lg:text-7xl font-bold text-white tracking-
tight leading-[1.1] mb-6 drop-shadow-2xl">
          {t("empoweringFarmers")} <br class="hidden md:block"/>
          {t("withSmart")} <span className="text-transparent bg-clip-text bg-
gradient-to-r from-green-400 via-emerald-300 to-teal-
400">{t("digitalSolutions")}</span>
        </h1>

        <p className="text-zinc-300 text-lg md:text-xl max-w-2xl leading-relaxed mb-
10 font-light drop-shadow-lg">
          {t("heroDesc")}
        </p>

        <div class="flex flex-col sm:flex-row gap-4 w-full md:w-auto">
          <button
            onClick={() => navigate('/assistant')}
            className="px-8 py-4 rounded-xl bg-gradient-to-r from-green-600 to-
emerald-500 text-white font-semibold text-lg flex items-center justify-center gap-2
hover:shadow-[0_0_25px_rgba(16,185,129,0.4)] transition-all duration-300 hover:-
translate-y-1"
          >
            <Mic size={20} /> {t("tryVoiceAssistant")}
          </button>
          <button
            onClick={() => navigate('/market')}
            className="px-8 py-4 rounded-xl bg-zinc-900/50 backdrop-blur-md border
border-zinc-700 hover:border-zinc-500 text-white font-semibold text-lg flex items-
center justify-center gap-2 transition-all duration-300 hover:bg-zinc-800"
          >
            {t("exploreMarkets")} <ArrowRight size={20} />
          </button>
        </div>
      </div>

      {/* Scroll Indicator */}
      <div class="absolute bottom-6 left-1/2 -translate-x-1/2 z-10 flex flex-col
items-center gap-2 animate-bounce">
```

```
        <span className="text-zinc-500 text-sm tracking-widest
uppercase">{t("scroll")}</span>
        <div className="w-[1px] h-8 bg-gradient-to-b from-zinc-500 to-
transparent"></div>
      </div>
    </div>

    {/* Features Showcase Section */}
    <div className="py-16 md:py-24 px-6 md:px-12 max-w-7xl mx-auto relative z-
10">
      <div className="text-center mb-16">
        <h2 className="text-3xl md:text-5xl font-bold text-white mb-
4">{t("everythingYouNeed")}</h2>
        <p className="text-zinc-400 max-w-2xl mx-auto text-
lg">{t("integratedTools")}</p>
      </div>

      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6">
        {[
          {
            title: t("aiVoiceAssistant"),
            desc: t("aiVoiceDesc"),
            icon: <Mic className="w-8 h-8 text-green-400" />,
            link: "/assistant",
          },
          {
            title: t("liveMarketPrices"),
            desc: t("liveMarketDesc"),
            icon: <LineChart className="w-8 h-8 text-emerald-400" />,
            link: "/market",
          },
          {
            title: t("weatherIntelligence"),
            desc: t("weatherIntelDesc"),
            icon: <CloudSun className="w-8 h-8 text-sky-400" />,
            link: "/weather",
          },
          {
            title: t("cropHealthPests"),
            desc: t("cropHealthDesc"),
            icon: <Leaf className="w-8 h-8 text-lime-400" />,
            // link: "/crop-health",
            link: "/pest-detect",
          },
```

```
          {
            title: "NASA Health Cockpit",
            desc: "Professional orbital telemetry and environmental analytics for your
farm.",
            icon: <Bot className="w-8 h-8 text-cyan-400" />,
            link: "/satellite-cockpit",
          }
        ].map((feature, idx) => (
          <div
            key={idx}
            onClick={() => navigate(feature.link)}
            className="group cursor-pointer bg-zinc-900/40 border border-zinc-800
hover:border-green-500/50 p-8 rounded-3xl transition-all duration-500 hover:bg-zinc-
800/80 hover:-translate-y-2 hover:shadow-[0_10px_40px_-
15px_rgba(16,185,129,0.3)]"
          >
            <div className="w-16 h-16 rounded-2xl bg-zinc-950 border border-zinc-800
flex items-center justify-center mb-6 group-hover:scale-110 transition-transform
duration-500">
              {feature.icon}
            </div>
            <h3 className="text-xl font-bold text-white mb-3 group-hover:text-green-
400 transition-colors">{feature.title}</h3>
            <p className="text-zinc-400 leading-relaxed text-sm md:text-base">
              {feature.desc}
            </p>
          </div>
        ))}
      </div>
    </div>

    {/* Value Proposition / Mission Section */}
    <div className="border-t border-zinc-900 bg-zinc-950">
      <div className="max-w-7xl mx-auto px-6 md:px-12 py-16 md:py-24 flex flex-col
lg:flex-row items-center gap-16">
        <div className="flex-1 space-y-8">
          <h2 className="text-3xl md:text-5xl font-bold text-white leading-tight">
            {t("platformBuiltFor")} <span className="text-transparent bg-clip-text bg-
gradient-to-r from-emerald-400 to-green-500">{t("futureOfFarming")}</span>
          </h2>
          <div className="space-y-6">
            {[
              t("mission1"),
              t("mission2"),
```

```
          t("mission3")
        ].map((text, i) => (
          <div key={i} className="flex items-start gap-4">
            <div className="flex-shrink-0 w-6 h-6 rounded-full bg-green-500/20 text-
green-400 flex items-center justify-center mt-1">
              ✓
            </div>
            <p className="text-zinc-300 text-lg">{text}</p>
          </div>
        ))}
      </div>
    </div>
    <div className="flex-1 relative w-full aspect-[4/3] rounded-3xl overflow-hidden
border border-zinc-800 shadow-2xl">
      {/* Visual representation card */}
      <div className="absolute inset-0 bg-gradient-to-br from-green-900/30 to-zinc-
900 flex items-center justify-center p-8">
        <div className="w-full max-w-md bg-zinc-950 border border-zinc-800
rounded-2xl p-6 shadow-2xl transform rotate-3 hover:rotate-0 transition-transform
duration-500">
          <div className="flex items-center justify-between border-b border-zinc-
800 pb-4 mb-4">
            <div className="flex items-center gap-3">
              <div className="w-10 h-10 rounded-full bg-emerald-500/20 flex items-
center justify-center">
                <Leaf className="text-emerald-400 w-5 h-5"/>
              </div>
              <div>
                <p className="text-white font-medium">{t("cropHealthStatus")}</p>
                <p className="text-xs text-zinc-500">{t("updatedJustNow")}</p>
              </div>
            </div>
            <span className="px-3 py-1 bg-green-500/20 text-green-400 rounded-full
text-xs font-bold uppercase tracking-wider">{t("optimal")}</span>
          </div>
          <div className="space-y-3">
            <div className="h-2 bg-zinc-800 rounded-full overflow-hidden"><div
className="w-[85%] h-full bg-gradient-to-r from-green-600 to-emerald-
400"></div></div>
            <div className="h-2 bg-zinc-800 rounded-full overflow-hidden"><div
className="w-[92%] h-full bg-gradient-to-r from-green-600 to-emerald-
400"></div></div>
```

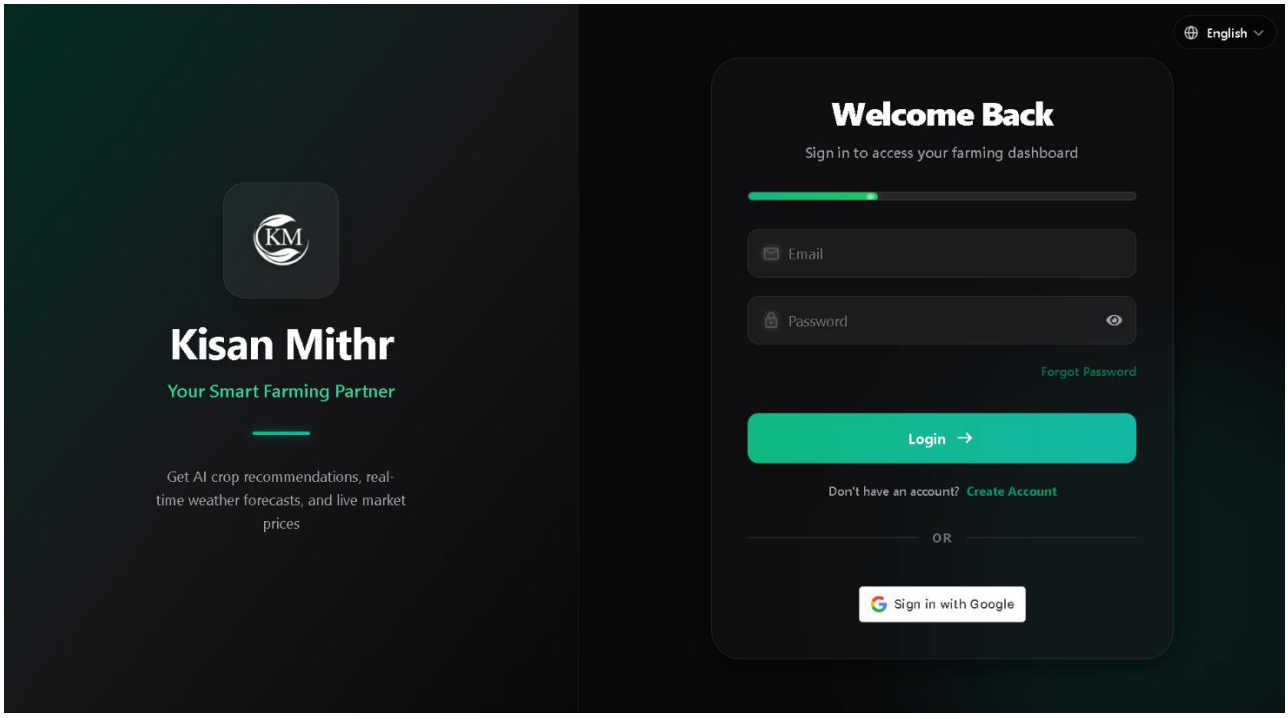
```
                <div className="h-2 bg-zinc-800 rounded-full w-2/3 overflow-
hidden"><div className="w-[70%] h-full bg-gradient-to-r from-green-600 to-
emerald-400"></div></div>
              </div>
            </div>
          </div>
        </div>
      </div>

      {/* Professional Feedback & Complaints Section */}
      <div className="relative border-t border-zinc-900 bg-zinc-950/50 py-16 md:py-
24 overflow-hidden">
        {/* Abstract background elements */}
        <div className="absolute top-0 right-0 w-96 h-96 bg-green-500/5 rounded-full
blur-3xl -translate-y-1/2 translate-x-1/2"></div>
        <div className="absolute bottom-0 left-0 w-64 h-64 bg-emerald-500/5 rounded-
full blur-3xl translate-y-1/2 -translate-x-1/2"></div>

        <div className="max-w-4xl mx-auto px-6 relative z-10">
          <div className="text-center mb-16">
            <div className="inline-flex items-center gap-2 px-3 py-1 rounded-full bg-
emerald-500/10 border border-emerald-500/20 text-emerald-400 text-xs font-bold
uppercase tracking-widest mb-4">
              <MessageSquare size={14} /> {t("feedback")}
            </div>
            <h2 className="text-3xl md:text-5xl font-bold text-white mb-4">
              {t("weValueYour")} <span className="text-transparent bg-clip-text bg-
gradient-to-r from-green-400 to-emerald-500">{t("thoughts")}</span>
            </h2>
            <p className="text-zinc-400 text-lg max-w-2xl mx-auto">
              {t("feedbackDesc")}
            </p>
          </div>

          <div className="bg-zinc-900/40 backdrop-blur-xl border border-zinc-800
rounded-3xl p-6 sm:p-8 md:p-12 shadow-2xl relative overflow-hidden group">
            {/* Glossy overlay */}
            <div className="absolute inset-0 bg-gradient-to-br from-white/5 to-
transparent opacity-0 group-hover:opacity-100 transition-opacity duration-700
pointer-events-none"></div>

            <form onSubmit={handleFeedbackSubmit} className="space-y-6 relative z-
10">
```

```
          <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
            <div className="space-y-2">
              <label className="text-zinc-400 text-sm font-medium ml-1 flex items-
center gap-2">
                <User size={14} /> {t("yourName")}
              </label>
              <input
                type="text"
                value={feedback.name}
                onChange={(e) => setFeedback({...feedback, name: e.target.value})}
                placeholder={t("namePlaceholder")}
                className="w-full bg-zinc-950/50 border border-zinc-800 rounded-xl px-
5 py-4 text-white focus:outline-none focus:ring-2 focus:ring-green-500/50
focus:border-green-500/50 transition-all placeholder:text-zinc-700"
              />
            </div>
            <div className="space-y-2">
              <label className="text-zinc-400 text-sm font-medium ml-1 flex items-
center gap-2">
                <Mail size={14} /> {t("yourEmail")}
              </label>
              <input
                type="email"
                defaultValue={user?.email || ""}
                disabled={!!user?.email}
                placeholder={t("emailPlaceholder")}
                className="w-full bg-zinc-950/50 border border-zinc-800 rounded-xl px-
5 py-4 text-white focus:outline-none focus:ring-2 focus:ring-green-500/50
focus:border-green-500/50 transition-all placeholder:text-zinc-700 disabled:opacity-50
disabled:cursor-not-allowed"
              />
            </div>
          </div>

          <div className="space-y-2">
            <label className="text-zinc-400 text-sm font-medium ml-1 flex items-center
gap-2">
              <MessageSquare size={14} /> {t("yourMessage")}
            </label>
            <textarea
              rows="5"
              value={feedback.message}
              onChange={(e) => setFeedback({...feedback, message: e.target.value})}
              placeholder={t("messagePlaceholder")}
```

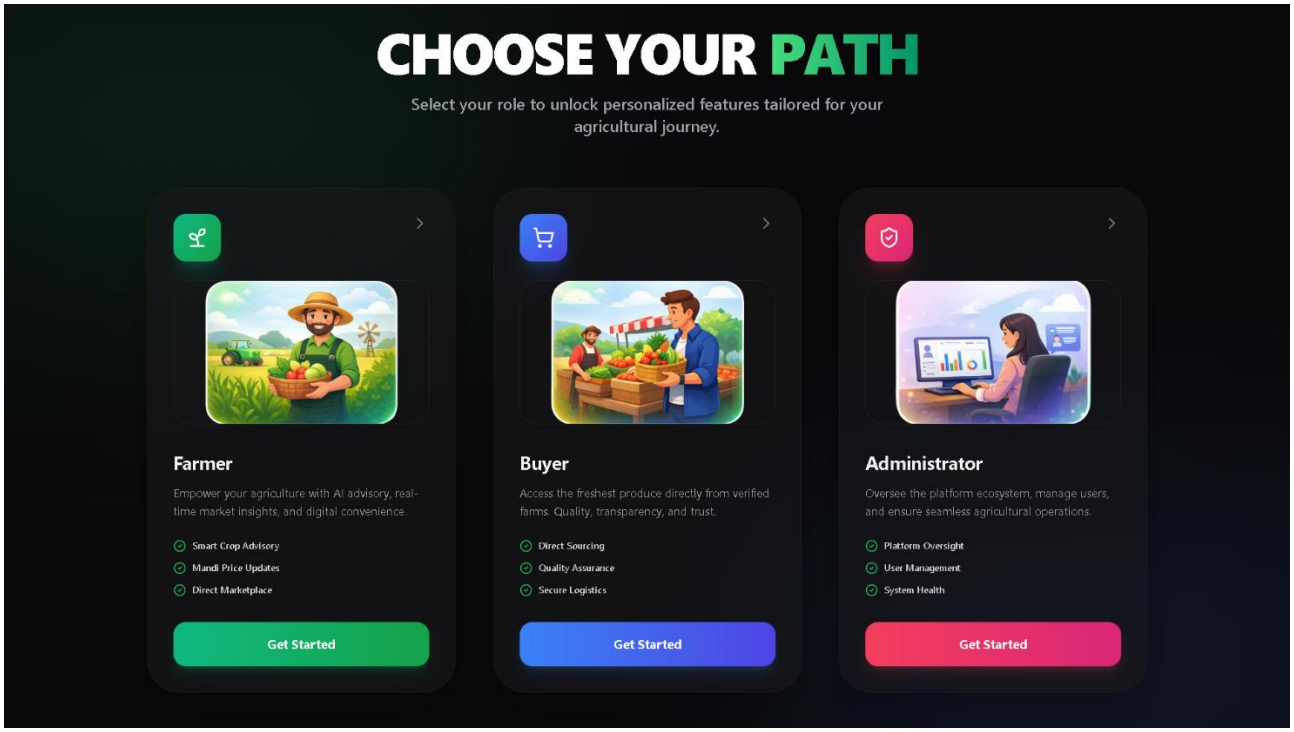
```
            className="w-full bg-zinc-950/50 border border-zinc-800 rounded-xl px-5
py-4 text-white focus:outline-none focus:ring-2 focus:ring-green-500/50 focus:border-
green-500/50 transition-all placeholder:text-zinc-700 resize-none"
          ></textarea>
        </div>

        <button
          type="submit"
          disabled={isSubmitting}
          className="w-full md:w-auto px-10 py-4 rounded-xl bg-gradient-to-r from-
green-600 to-emerald-500 text-white font-bold text-lg flex items-center justify-center
gap-3 hover:shadow-[0_0_30px_rgba(16,185,129,0.4)] transition-all duration-300
hover:-translate-y-1 disabled:opacity-50 disabled:transform-none select-none"
        >
          {isSubmitting ? (
            <div className="w-6 h-6 border-2 border-white/30 border-t-white
rounded-full animate-spin"></div>
          ) : (
            <>
              <Send size={20} /> {t("sendFeedback")}
            </>
          )}
        </button>
      </form>
    </div>
  </div>
</div>

{/* Professional Footer */}
<Footer />

</div>
);
};

export default HomePage;
```

SCREEN SHOTS

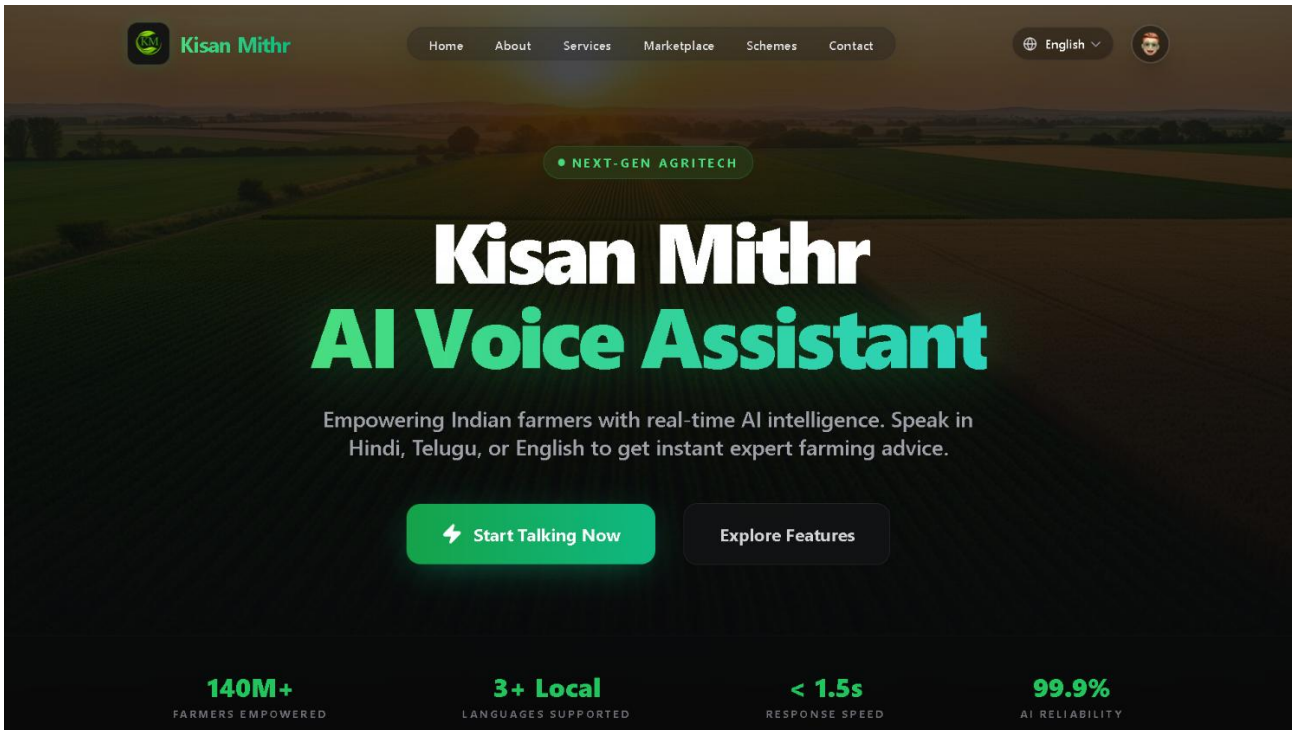
Kisan Mithr Login Page:



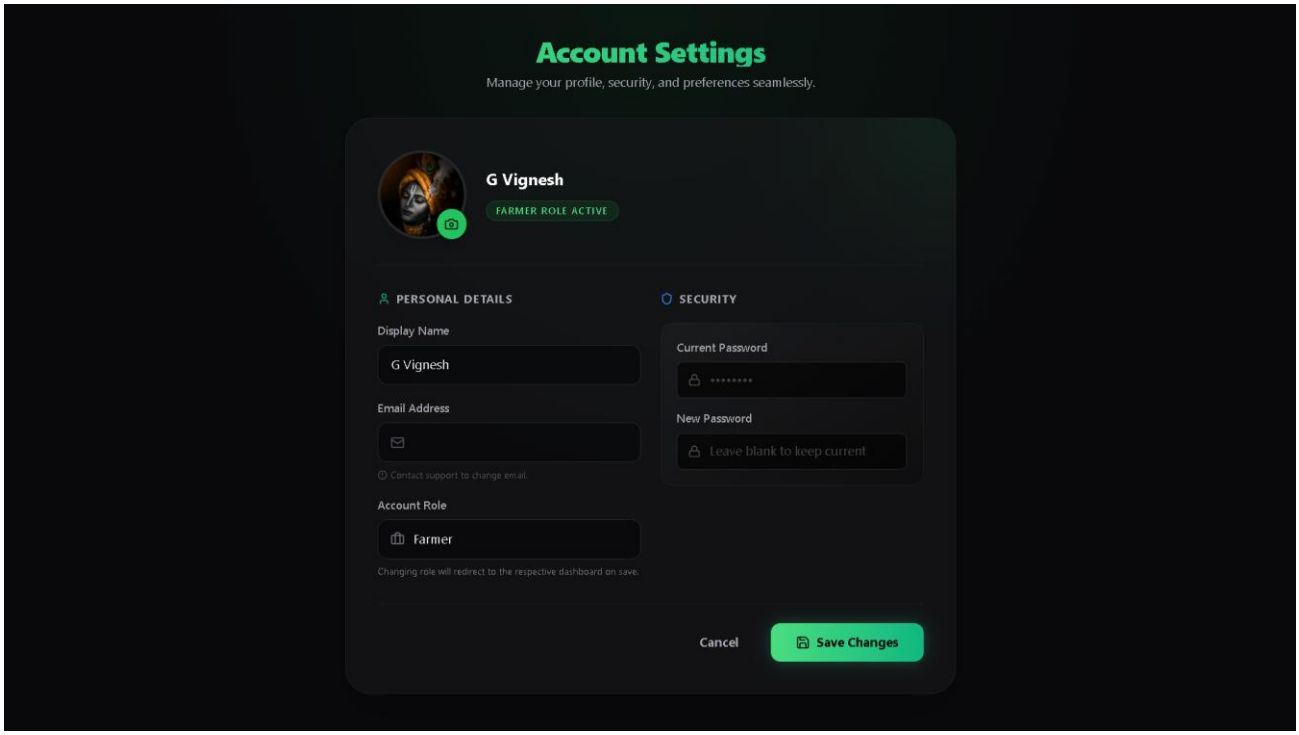
Role Selection:



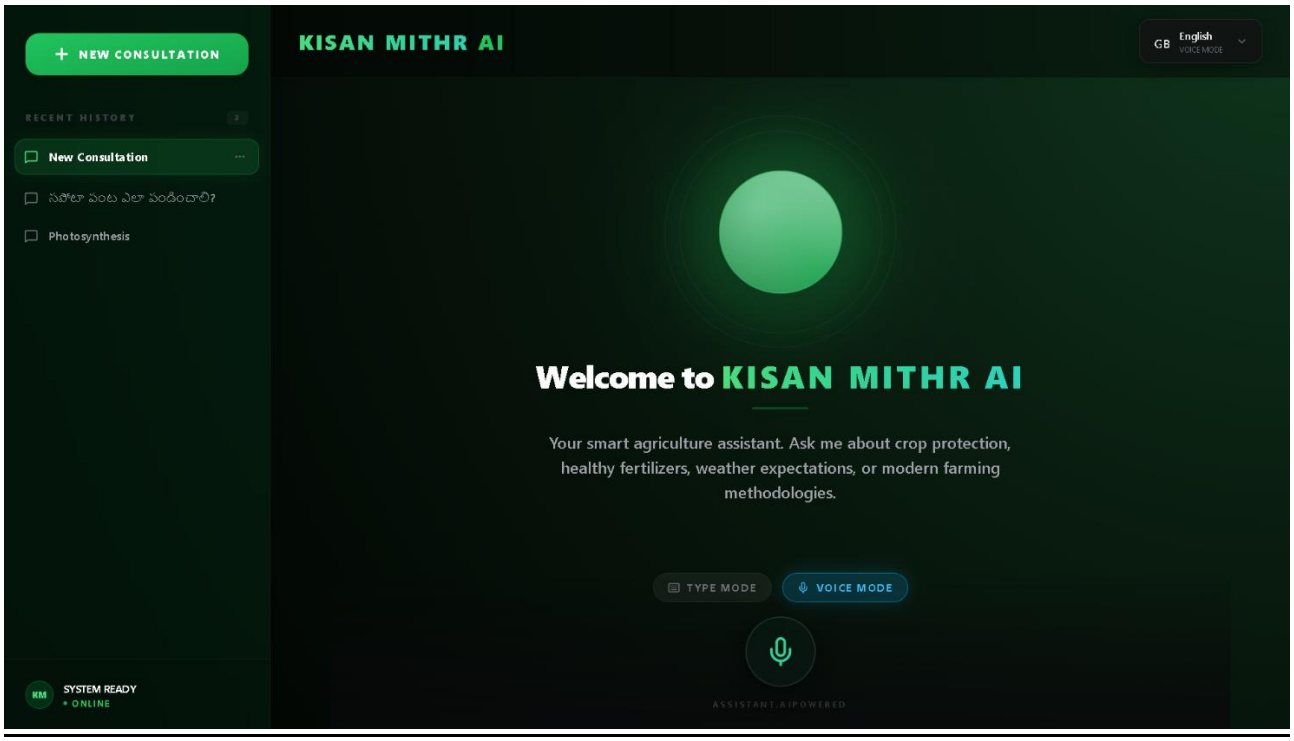
HomePage Dashboard:



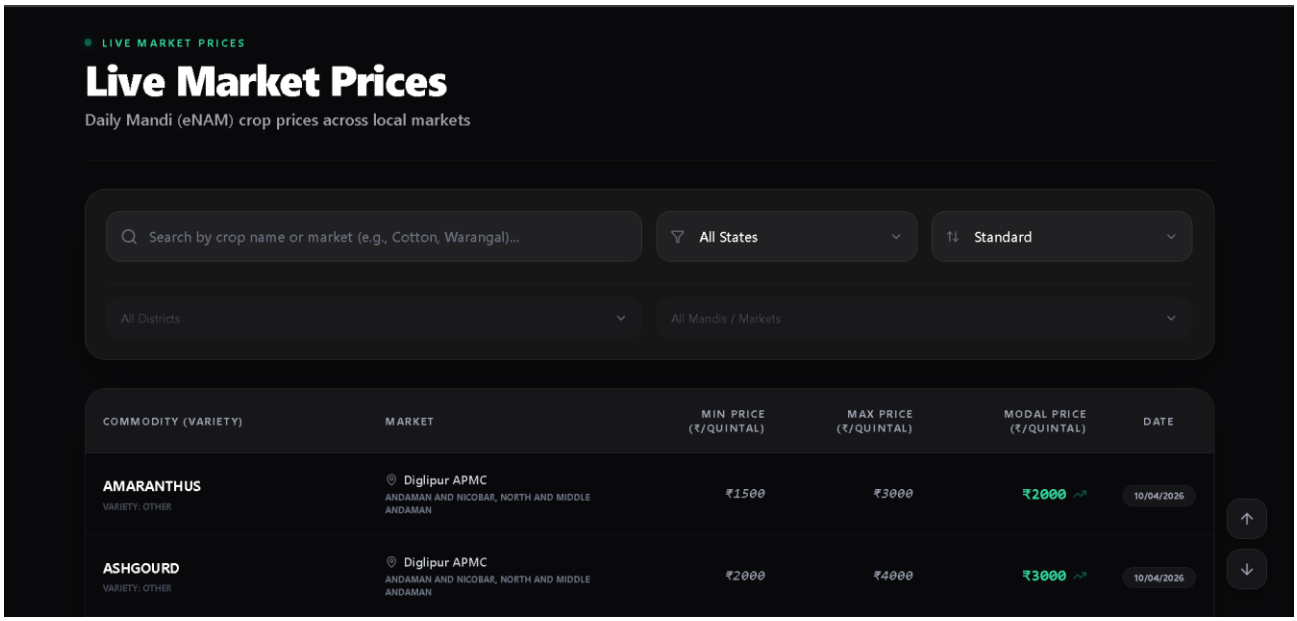
Accounts Page:



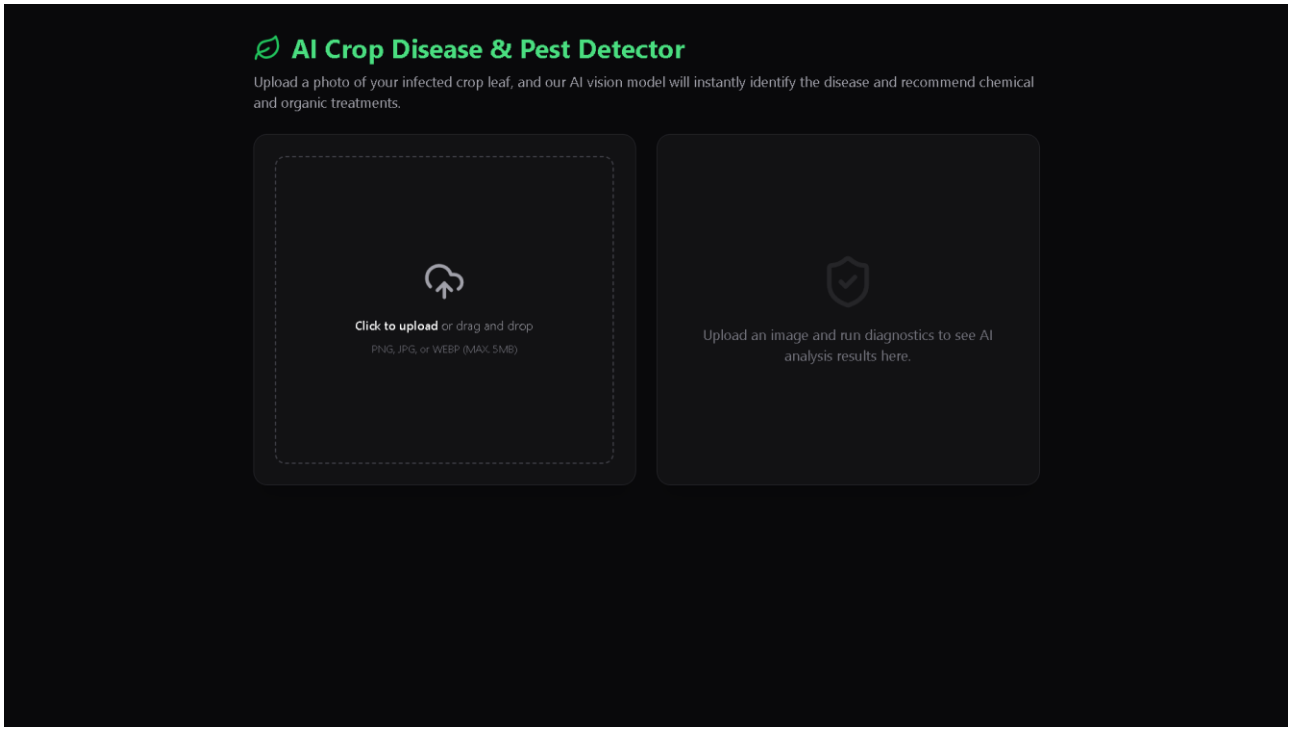
AI Assistant Page:



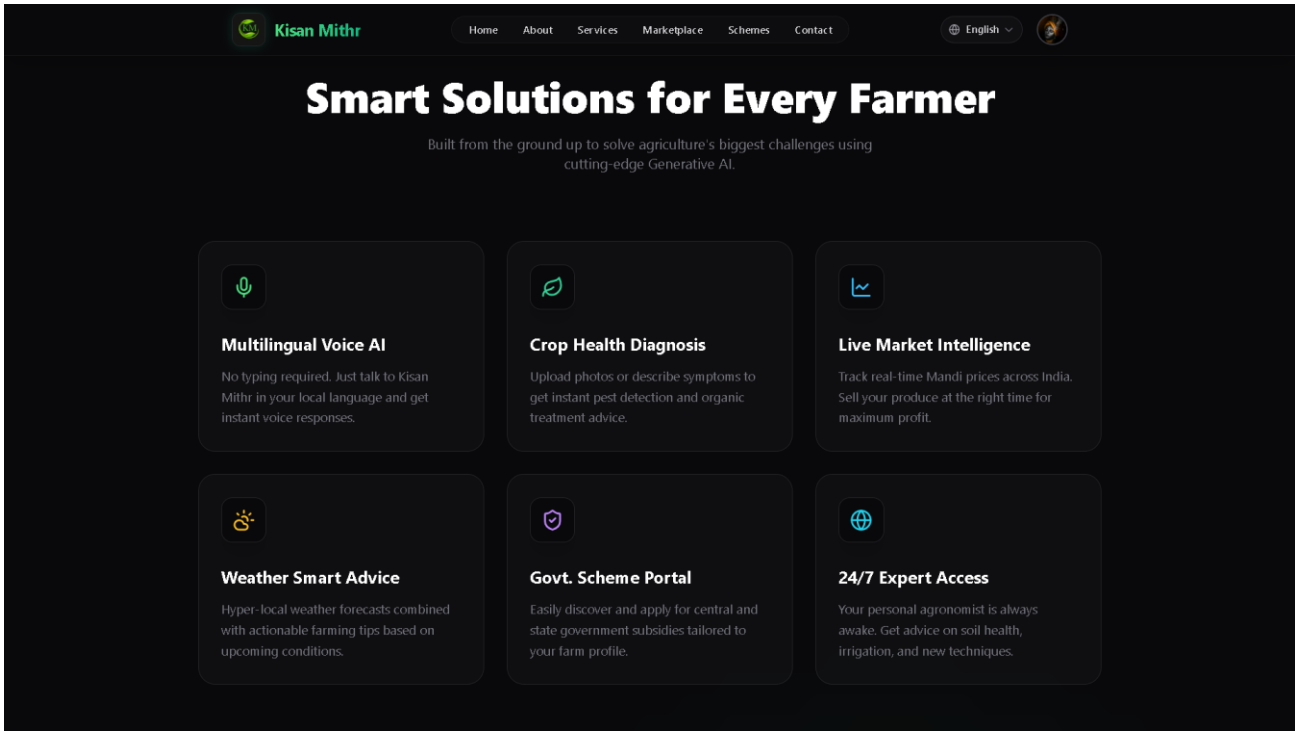
Market Prices:



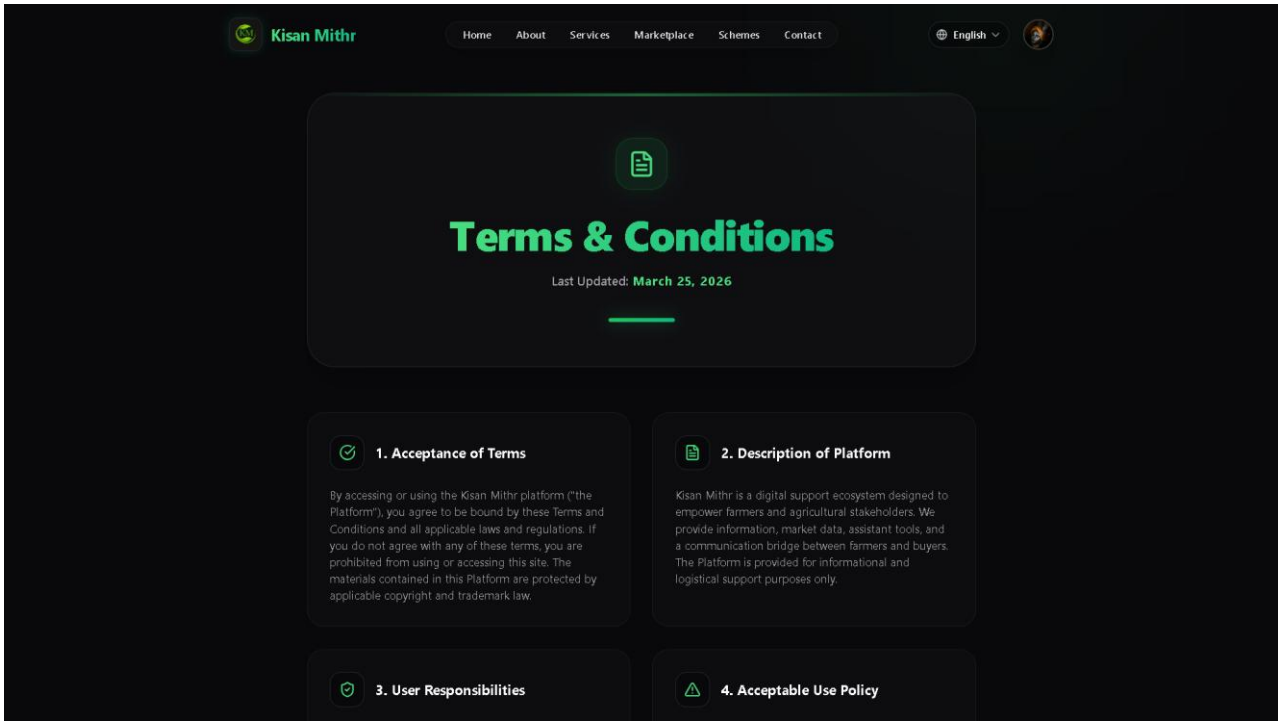
AI Pest Detector:



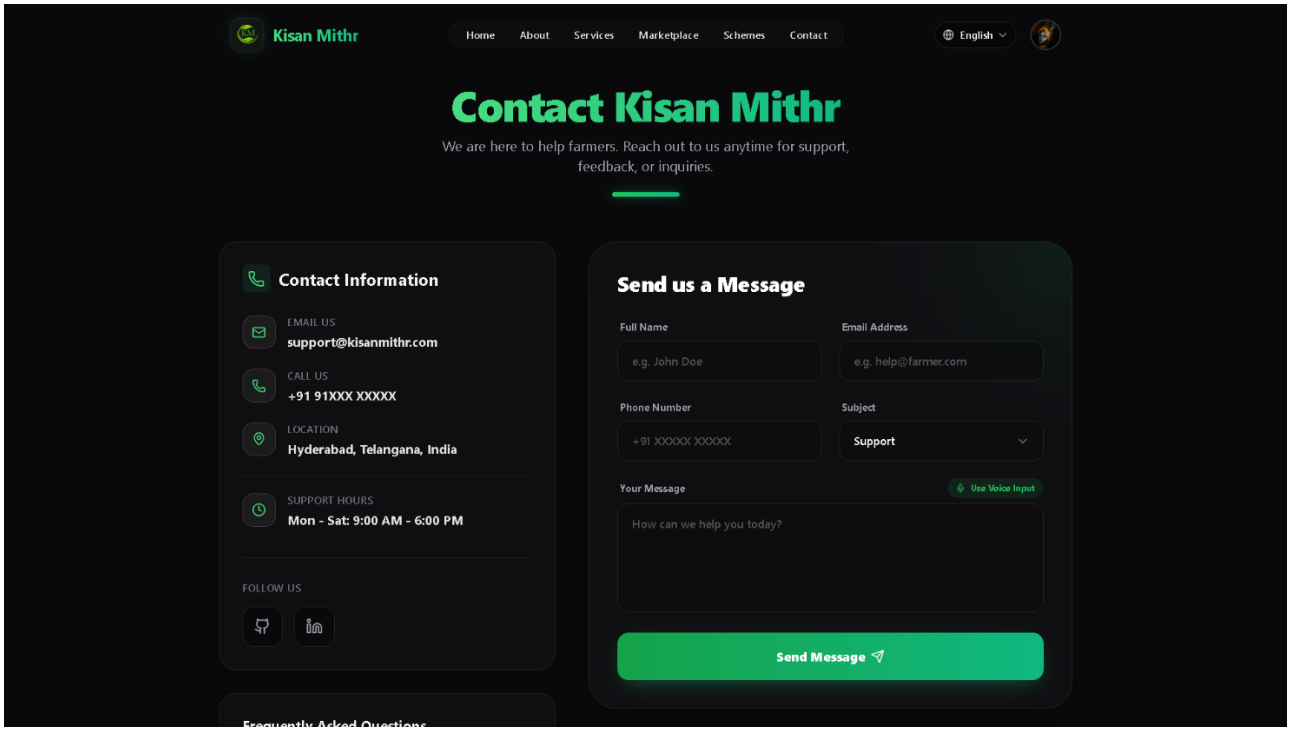
Features:



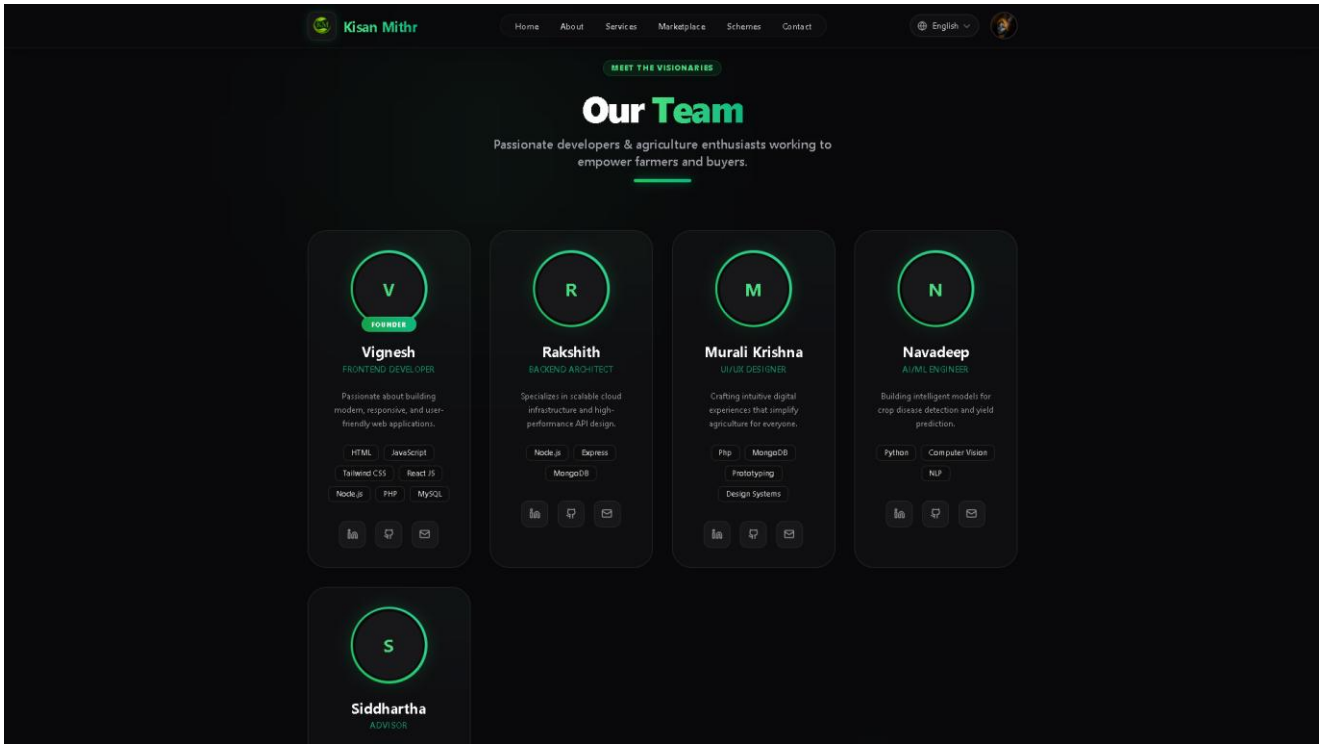
Terms & Conditions:



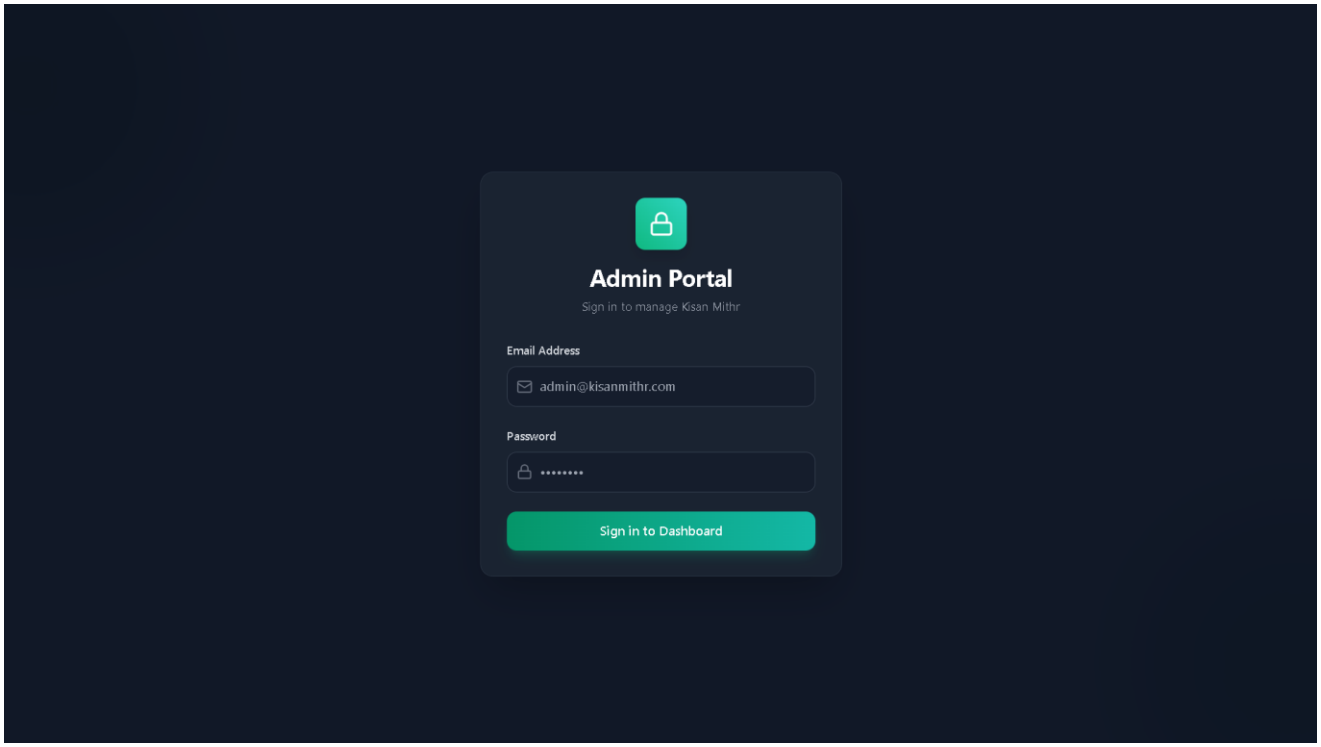
Contact Page:



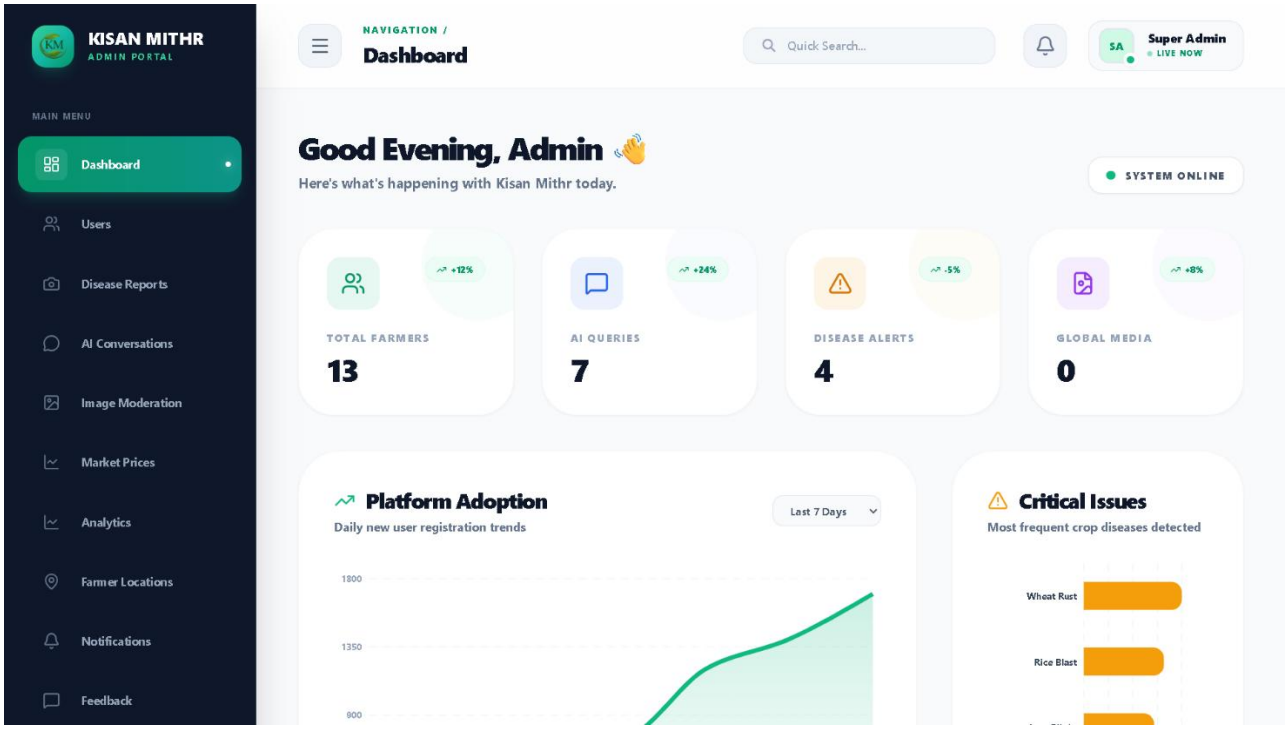
Our Team Members:



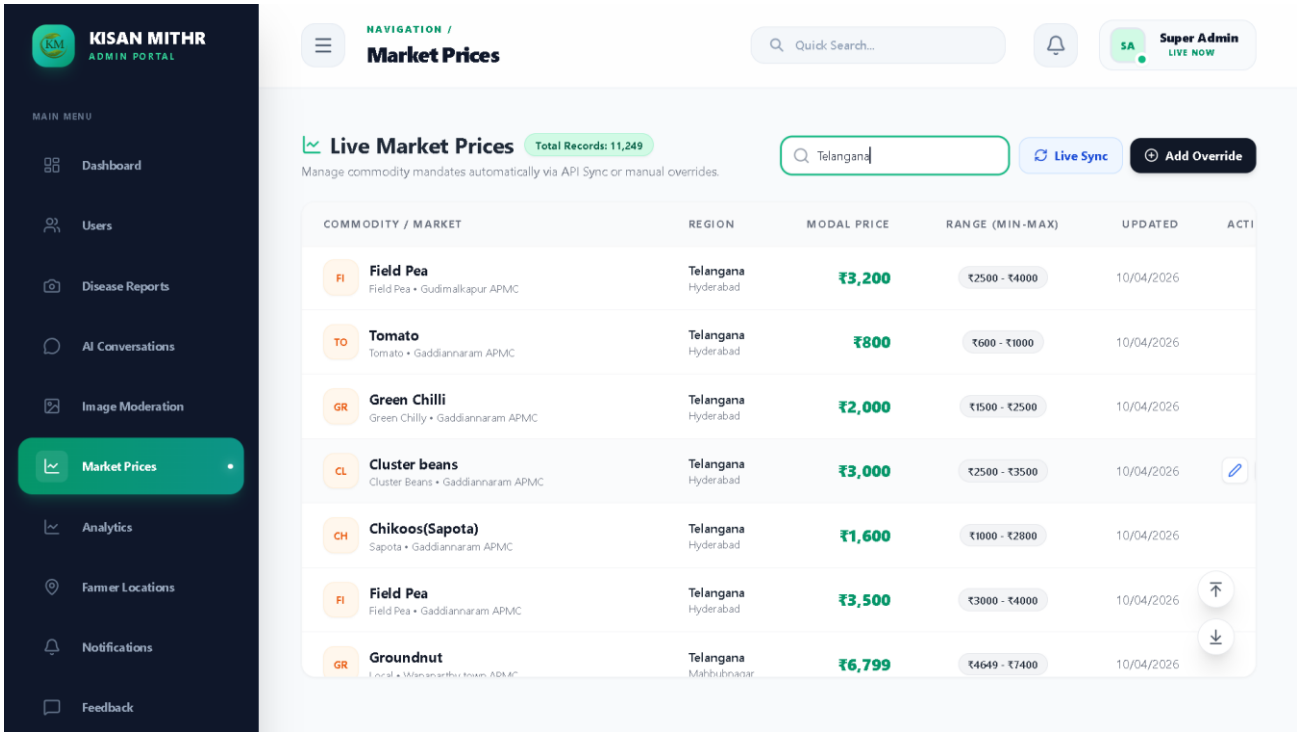
ADMIN Login Page:



ADMIN Dashboard:



ADMIN Market Update:



TESTING

TESTING

The **Kisan Mithr platform** was thoroughly tested to ensure that all farmer-side features function correctly, efficiently, and reliably. Testing covered the complete user journey, starting from user registration and login (including email/password, Google OAuth, and OTP verification) to the usage of all major modules such as AI Voice Assistant, Weather Intelligence, Live Market Prices, Crop Disease Detection, Farm Profile, Livestock Guide, and Government Schemes. Both **valid and invalid input scenarios** were tested across all modules to ensure proper validation, error handling, and system stability.

1. Unit Testing

Unit testing focuses on testing individual components of the system to ensure they work correctly.
Examples include login validation, OTP verification, image upload checks, and form handling.

2. Integration Testing

Integration testing ensures that different modules of the system work together properly. It verifies the connection between frontend (React.js), backend (Node.js), database (MongoDB), and external APIs like AI and weather services.

3. System Testing

System testing validates the complete application as a whole. It checks the full workflow such as user registration, login, accessing features like AI assistant, weather updates, and crop detection.

4. Security Testing

Security testing ensures that the application is safe and protects user data. It includes password encryption, authentication checks, OTP validation, and secure access to protected pages.

5. Output Testing

Output testing focuses on verifying whether the system produces the **correct and expected results** for given inputs. It ensures that all outputs displayed to the user are accurate, meaningful, and properly formatted. This testing ensures that users (farmers) receive **reliable and accurate information**, which is critical for decision-making.

CONCLUSION

CONCLUSION

The Kisan Mithr system is a smart agricultural platform designed to assist farmers in making informed decisions using modern technologies such as Artificial Intelligence, real-time weather analysis, and market data insights. It offers a wide range of features including crop recommendations, disease detection, live market prices, weather updates, and government scheme guidance, all through a simple and user-friendly interface. The system ensures accessibility for farmers with different levels of technical knowledge and supports multiple languages for better usability. Extensive testing confirmed that all modules function correctly, providing accurate and reliable results.

Overall, Kisan Mithr acts as a reliable digital assistant for farmers, improving productivity and reducing risks. It has strong potential to support the future of smart farming.